Mon premier projet Symfony

Table des matières

Démarrage : 2

Démarrer le serveur : 2

Le MakerBundle 5

TWIG, le moteur de Templates 7

- Le principe fondamental : l'héritage 7
- Les filtres (ou les Pipes) 8
- Les opérateurs logiques 8
- Inclusion de templates 9
- En résumé : 9

Sass… ? 12

Démarrage :

Dès le début, on a déjà le choix. On peut en effet créer des applications / sites web avec **différents niveaux de complexité** avec Symfony. Par exemple, si l'on veut créer simplement une API, nous n'allons pas avoir besoin d'importer TWIG, le moteur de template de Symfony. Et si on veut monter un site web complet, alors nous allons avoir besoin de lui préciser.

Nous allons donc exécuter la commande suivante :

```
symfony new premier_projet --version="7.4.*" --webapp
```

Questions :

- Pourquoi préciser la version ? Parce que la version 8 de Symfony est sortie mais qu'elle n'est pas encore considérée comme étant la version principale, la version LTS (Long Term Support). En effet, la version 8 est encore sujette à des modifications à la volée, la LTS sera la version 8.4. En revanche, la 7.4, en plus d'être majoritairement utilisée, a une fin de support annoncée en 2028 / 2029, nous pouvons donc rester sur ce que l'on va considérer comme la dernière version stable de Symfony.
- Pourquoi lui préciser --webapp ? Parce que l'on va vouloir importer le moteur de template de Symfony ainsi que toutes ses dépendances puisque l'on va créer un site complet. Si on n'a besoin que d'une API et pas de visuel, on peut alors retirer le flag --webapp de la commande.

Structure :

```
premier_projet/
├── bin/          # Commandes (bin/console — important, on y reviendra vite)
├── config/       # Configuration des services, routes, packages
├── public/       # Seul dossier accessible depuis le web (index.php est le point d'entrée de l'application)
├── src/          # Votre code PHP : Controllers, Entities, Services...
│   └── Controller/
│   └── Entity/
│   └── Repository/
├── templates/    # Vues Twig
├── var/          # Cache et logs (généré automatiquement)
├── vendor/       # Dépendances installées par Composer
└── .env          # Variables d'environnement (base de données, etc.)
```

Démarrer le serveur :

Avant de démarrer le serveur, on va installer un **certificat** local. Pourquoi ? Parce que Symfony préfère fonctionner en HTTPS, et qu'il va nous mettre un petit avertissement à chaque fois qu'on démarre le serveur ☺ Alors on va prendre les devants et installer le certificat comme ceci :

```
symfony server:ca:install
```

Une fois ce certificat généré, on va devoir le trouver, pour cela utiliser la commande :

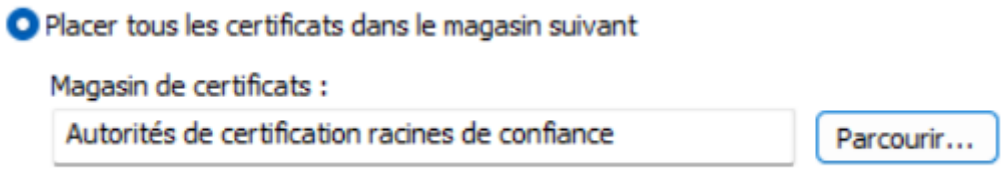
```
find ~ -name "rootCA.pem" 2>/dev/null
```

Puis on va copier ce certificat dans la couche Windows, avec la commande suivante :

```
cp ~/.symfony5/certs/rootCA.pem /mnt/c/Users/VOTRE_NOM/Desktop/symfony-ca.pem
```

Sur votre bureau, vous allez retrouver le certificat en question. On va alors le renommer en **.crt** et double-cliquer dessus, puis choisir « Installer le certificat », pour un « Ordinateur local ».

Vous allez ensuite choisir « Placer tous les certificats dans le magasin suivant », puis cliquer sur « Parcourir » juste à côté et choisir « Autorités de certification racines de confiances ».



Et une fois que le certificat est installé, on redémarre Chrome, et dans notre dossier premier_projet, on va pouvoir exécuter :

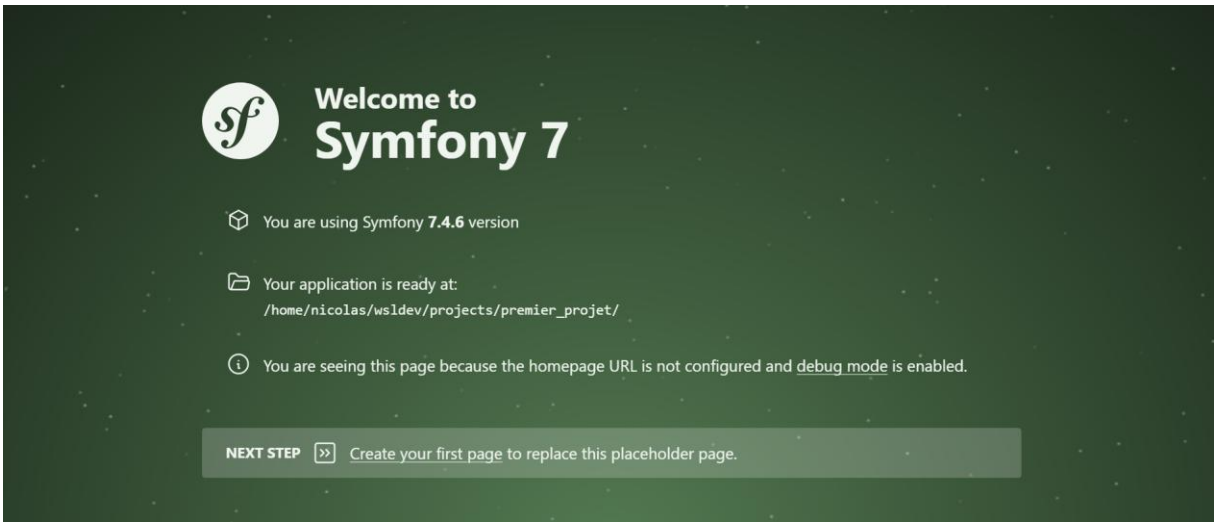
```
symfony serve
```

Et on se retrouve avec un serveur de développement démarré et accessible sur https://localhost:8000

Si on veut faire plus simple, on peut aussi lui dire de ne pas s'embêter avec le HTTPS en lui donnant la commande :

```
symfony serve --no-tls
```

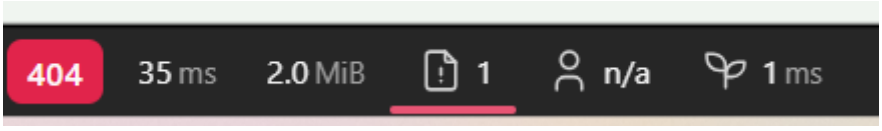
Avant de passer à notre création, un mot sur ce qu'on voit à l'écran :



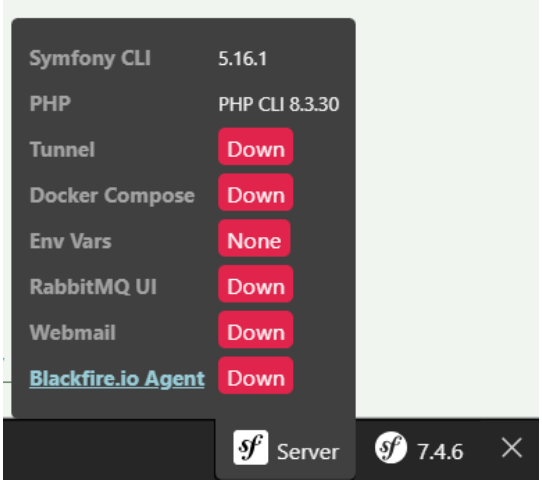
Un message d'accueil simple qui vous permet de vérifier que tout fonctionne bien : Le numéro de version, le chemin du projet, bref.

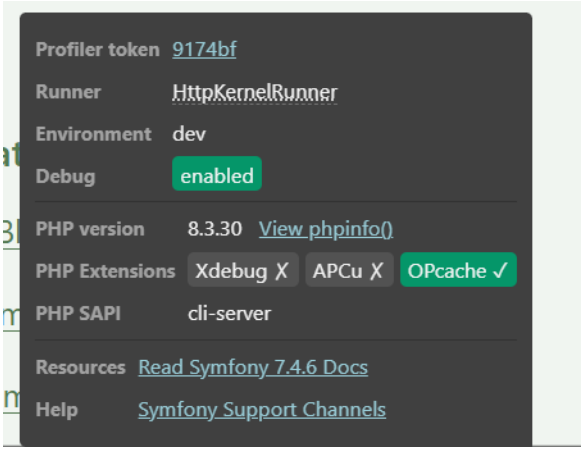
Des métriques :

Respectivement le statut, le temps de chargement, le poids de la page, la console (avec une erreur ici), l'utilisateur connecté si connexion il y a, et la stack trace du template Twig affiché.



À droite, on retrouve l'état des différents services du serveur, et la stack installée :





Le MakerBundle

Tout comme avec le CLI Angular, Symfony dispose d'outils nous permettant de **générer du code** plus facilement qu'à la main. Cet outil s'appelle le **MakerBundle**, et il doit être installé avec Composer, comme ceci :

```
composer require symfony/maker-bundle --dev
```

Le flag --dev sert à faire du **MakerBundle** une dépendance seulement lorsque l'on est en environnement de développement. Effectivement, on ne va pas l'installer en production puisqu'on ne génère pas de code en production.

Une fois ce bundle installé, on va pouvoir l'utiliser avec l'outil **bin/console** que l'on a aperçu plus haut : Ce fichier binaire est un **exécutable** qui va nous permettre de lancer un tas de commandes indispensables. Voici donc comment nous allons créer notre nouveau Contrôleur :

```
php bin/console make:controller AccueilController
```

Un peu à la façon de **ng generate component monComposant**, le MakerBundle va nous créer deux fichiers :

- Un Contrôleur, que l'on va retrouver dans src/Controller
- Un Template, que l'on va retrouver dans templates/accueil

Dans le contrôleur, on retrouve la méthode **render** qui est appelée, en passant en paramètre le **template** à rendre, avec un certain nombre de paramètre comme « component_name »

Voici un résumé de tout ce qui se joue ici :

src > Controller > AccueilController.php

```php
<?php

namespace App\Controller;
// Comme les imports dans Angular, on inclut ici tout ce dont Symfony a besoin
// Abstract Controller est un contrôleur "généraliste" qui n'est jamais instancié mais qui
// contient des méthodes que tous les controlleurs "enfants" peuvent utiliser
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
// HttpFoundation est plus ou moins l'équivalent de HttpClient chez Angular. On utilise sa classe "Response"
// pour typer la valeur de retour de la fonction index()
use Symfony\Component\HttpFoundation\Response;
// Ici on importe la classe Route pour typer les attributs de la méthode.
use Symfony\Component\Routing\Attribute\Route;

final class AccueilController extends AbstractController
{
    // Dans d'anciennes versions de Symfony, on décrivait toutes les routes de l'Appli dans un fichier YAML
    // Bien que toujours supporté, maintenant Symfony préconise de décrire les routes comme ceci directement dans les contrôleurs
    // Voyez ça un peu comme un décorateur Angular
    #[Route('/accueil', name: 'app_accueil')]
    public function index(): Response
    {
        // On appelle la méthode render de AbstractController pour orienter vers la vue en question.
        return $this->render('accueil/index.html.twig', [
            'controller_name' => 'AccueilController'
        ]);
    }
}
```

Et du côté du template, on a ceci :

templates > accueil > index.html.twig

```
{% extends 'base.html.twig' %}

{% block title %}Hello AccueilController!{% endblock %}

{% block body %}

<style>
    .example-wrapper { margin: 1em auto; max-width: 800px; width: 95%; font: 18px/1.5 sans-serif; }
    .example-wrapper code { background: #F5F5F5; padding: 2px 6px; }
</style>

<div class="example-wrapper">
    <h1>Hello {{ controller_name }}! ✅</h1>

    This friendly message is coming from:
    <ul>
        <li>Your controller at <code>/home/nicolas/wsldev/projects/premier_projet/src/Controller/AccueilController.php</code></li>
        <li>Your template at <code>/home/nicolas/wsldev/projects/premier_projet/templates/accueil/index.html.twig</code></li>
    </ul>

</div>
{% endblock %}
```

On peut y voir que le paramètre « controller_name » passé dans la fonction **render** est bien accessible ici directement dans le template, avec de la **string interpolation**. D'ailleurs on peut passer les paramètre que l'on souhaite depuis le contrôleur vers le template. Par exemple :

```
#[Route('/accueil', name: 'app_accueil')]
public function index(): Response
{
    // On appelle la méthode render de AbstractController pour orienter vers la vue en question.
    return $this->render('accueil/index.html.twig', [
        'controller_name' => 'AccueilController',
        'message_accueil' => 'Salut la classe 8) et bienvenue !'
    ]);
}
```

message_accueil devient accessible dans accueil/index.html.twig :

```
<div class="example-wrapper">
    <h1>Hello {{ controller_name }}! ✅</h1>
    {{ message_accueil }}
```

Attardons nous un petit peu sur TWIG avant d'aller plus loin.

TWIG, le moteur de Templates

Le principe fondamental : l'héritage

Twig fonctionne sur un système d'**héritage de templates**. L'idée est simple : vous définissez un template parent qui contient la structure HTML commune à toutes vos pages (header / menu, footer, etc…) et chaque page enfant **hérite** de ce parent et vient remplir des zones prédéfinies appelées **blocs**. Chaque bloc est donc une sorte **d'emplacement prévu** pour le contenu d'un autre template.

C'est exactement le même principe que l'héritage en POO, une classe enfant hérite d'une classe parent et surcharge certaines méthodes.

Si on regarde dans le template parent par exemple, base.html.twig, on retrouve ceci :

```
<body>
    {% block body %}{% endblock %}
</body>
```

C'est la déclaration du block **body**. Attention je ne parle pas de la balise ici, mais du nom donné au BLOC.

Ce bloc est une zone que les templates enfants pourront venir **remplacer**. Et c'est d'ailleurs exactement ce qui se passe dans notre cas : On a le block body déclaré également dans le template accueil/index.html.twig. Il va aller se placer dans l'emplacement prévu par le template parent.

La **filiation** entre les deux templates est déclarée ici :

```
templates > accueil > index.html.twig
1   {% extends 'base.html.twig' %}
```

Et si je ne veux pas **remplacer le contenu** du block du parent, mais simplement y **rajouter quelque chose**, alors je vais pouvoir avec **parent()**. Par exemple si dans mon accueil je veux inclure une feuille de style supplémentaire :

```
{% block stylesheets %}
    {{ parent() }}
    {# Ajoute une feuille de style en plus de celles du parent #}
    <link rel="stylesheet" href="/css/accueil.css">
{% endblock %}
```

Les filtres (ou les Pipes)

Comme dans Angular, TWIG fonctionne avec des outils de formattage qui permettent de transformer une valeur à l'affichage mais SANS LA MODIFIER. Voici une liste non exhaustive :

```
{# Mettre en majuscules #}
{{ nom|upper }}

{# Tronquer un texte à 100 caractères #}
{{ description|truncate(100) }}

{# Formater une date #}
{{ article.createdAt|date('d/m/Y') }}

{# Valeur par défaut si la variable est nulle #}
{{ surnom|default('Anonyme') }}
```

Les opérateurs logiques

Comme dans tous les langages, on retrouve des opérateurs logiques de **conditions** et de **boucles**.

Ils se notent avec la syntaxe {% opérateur %}, là où dans les templates Angular on les préfixait avec une arobase.

```
{# Condition #}
{% if user is defined and user.age >= 18 %}
    <p>Bienvenue, {{ user.prenom }} !</p>
{% elseif user is defined %}
    <p>Accès réservé aux majeurs.</p>
{% else %}
    <p>Veuillez vous connecter.</p>
{% endif %}

{# Boucle #}
{% for produit in produits %}
    <div class="produit">
        <h2>{{ produit.nom }}</h2>
        <p>{{ produit.prix }} €</p>
    </div>
{% else %}
    {# Le bloc else s'affiche si la liste est vide #}
    <p>Aucun produit disponible.</p>
{% endfor %}
```

Petite particularité ici : la boucle {% for %} possède un bloc {% else %}. Cela nous évite d'avoir a tester si mon tableau est vide ou non, je peux gérer le cas plus facilement ainsi.

Inclusion de templates

Il est possible de vouloir inclure un template dans un autre sans vouloir passer par la logique de bloc. Par exemple je veux afficher un formulaire d'inscription sur une page de mon site, mais je ne veux l'afficher que si l'utilisateur n'est pas déjà connecté !

Mon cas de figure comportant une exception, je peux alors conditionner l'inclusion d'une sous-partie d'un template, et l'inclure comme ceci :

```
{# Boucle #}
{% for produit in produits %}
    {% include 'components/_carte_produit.html.twig' with { produit: produit }
%}
{% endfor %}
```

Et dans mon template components/_carte_produit.html.twig, j'aurais ceci :

```
{# templates/components/_carte_produit.html.twig #}
<div class="carte">
    <h2>{{ produit.nom }}</h2>
    <p>{{ produit.prix }} €</p>
</div>
```

Deux remarques ici : Le template qui va être inclus est **préfixé d'un _**

C'est une norme que Symfony préconise, il est ainsi très facile d'identifier les templates qui sont des parties d'un autre.
Enfin, on peut voir qu'avec le mot-clé **with**, je lui passe un paramètre accessible à ce template. Très utile pour faire descendre des données jusqu'à leur template.

En résumé :

Syntaxe	Rôle
`{{ variable }}`	Afficher une valeur
`{% tag %}`	Logique (if, for, block...)
`{# commentaire #}`	Commentaire (non rendu dans le HTML)
`{% extends 'base.html.twig' %}`	Hériter d'un parent
`{% block nom %}{% endblock %}`	Définir une zone remplaçable
`{{ parent() }}`	Réutiliser le contenu du bloc parent
`{% include 'fichier.twig' %}`	Inclure un sous-template
`|filtre`	Transformer une valeur

BONUS : Les Extensions TWIG

Twig est un composant injecté par Symfony, il est donc accessible partout. On peut vouloir que Twig ait accès en **permanence** à des données, peu importe si on lui les passe ou non. Par exemple si je veux générer des liens de menus dynamiques, disons une liste de catégories de produits pour faire un dropdown, il ne faut surtout pas que je m'amuse à passer mes catégories en tant que paramètre à chaque Route que je crée ! Je vais forcément finir par en oublier une, et puis si je dois changer quelque chose, il faudra répercuter cette modif partout, comme d'habitude.

On va donc utiliser le **MakerBundle** pour créer une **extension twig**

```
php bin/console make:twig-extension AppExtension
```

Puis on va se rendre dans **src/Twig/AppExtension.php**

Ici, on va avoir besoin du **Repository** de l'entité que l'on veut passer en paramètre, et on va s'en servir pour passer les objets dans une fonction **getGlobals** (vous vous souvenez du tableau $_GLOBALS en PHP simple ? C'est la même idée)

Ça nous donne un fichier comme ceci si on reste sur mon exemple des catégories :

```php
<?php

namespace App\Twig\Extension;

use App\Repository\CategorieRepository;
use Twig\Extension\AbstractExtension;
use Twig\Extension\GlobalsInterface;

// On n'oublie pas d'hériter de la classe GlobalsInterface si on veut utiliser les GLOBALS
class AppExtension extends AbstractExtension implements GlobalsInterface
{
    // Injection du Repository correspondant
    public function __construct(private CategorieRepository $categorieRepository) {}

    // Puis on surcharge la fonction getGlobals en lui passant ce que l'on veut dedans.
    public function getGlobals() :array {
        return [
            'categories_menu' => $this->categorieRepository->findAll()
        ]
    }
}
```

Et le résultat de tout ça, c'est quoi ? Eh bien tout simplement, vu que TWIG est injecté par Symfony de manière autonome, et que l'on vient de lui ajouter une extension, on a donc accès à la variable categories_menu dans tous les templates TWIG de l'appli, sans aucune action supplémentaire. Je peux donc sereinement faire mon menu :

EXERCICE

1. Utiliser le MakerBundle pour créer un Contrôleur LivreController
2. Dans ce contrôleur, on va lui créer une propriété statique **$livres**, qui va contenir un tableau de livres :

```php
private array $livres = [
    1 => [
        'titre'  => 'Le Seigneur des Anneaux',
        'auteur' => 'J.R.R. Tolkien',
        'annee'  => 1954,
        'genre'  => 'Fantasy',
        'resume' => 'Un hobbit part à l\'aventure pour détruire un anneau maléfique.',
    ],
    2 => [
        'titre'  => 'Dune',
        'auteur' => 'Frank Herbert',
        'annee'  => 1965,
        'genre'  => 'Science-Fiction',
        'resume' => 'Sur une planète désertique, un jeune noble affronte son destin.',
    ],
    3 => [
        'titre'  => '1984',
        'auteur' => 'George Orwell',
        'annee'  => 1949,
        'genre'  => 'Dystopie',
        'resume' => 'Dans un monde totalitaire, un homme tente de résister au système.',
    ],
];
```

3. Passer la propriété **$livres** depuis le contrôleur vers le template grâce à la méthode **render**
4. Dans le template, on a deux blocks à remplacer du template parent : le bloc **title** et le block **body**
5. Dans le template, utiliser une boucle {% for %} pour lister tous les ouvrages en récupérant la variable passée par le contrôleur. On n'affichera que l'auteur, l'année et le genre
6. Une fois que tout ça fonctionne, on va créer une deuxième Route dans notre contrôleur. La route de la vue « Détail ». Cette route devra s'écrire comme ceci :

```php
#[Route('/livres/{id}', name: 'app_livre_detail')]
public function detail(int $id): Response
{
    // Si l'id n'existe pas, on renvoie une 404
    if (!isset($this->livres[$id])) {
        throw $this->createNotFoundException('Ce livre n\'existe pas.');
    }

    return $this->render('livre/detail.html.twig', [
        'livre' => $this->livres[$id],
    ]);
}
```

7. Il faut maintenant créer le template detail.html.twig, qui contiendra toutes les informations du livre : son auteur et son genre, son année, mais surtout son résumé qui ne figurait pas dans la liste. Vérifiez que votre vue fonctionne en appelant l'URL /livres/1 par exemple. Là aussi on doit remplacer le bloc **title** et le block **body** du parent.
8. Une fois que cela fonctionne nous allons faire communiquer ces deux pages l'une avec l'autre en créant des liens **<a>**. Mais avec Symfony on ne met pas simplement des liens <a> avec un href, on peut passer directement les **noms** des routes**, c'est-à-dire les attributs name** au dessus de chaque méthode du contrôleur. On fait ceci pour une bonne raison : Si pour des raisons de référencement on souhaite changer le lien qui renvoie vers notre vue, on n'aura qu'à le faire une fois dans le contrôleur et pas à chaque fois que le lien est appelé, car TWIG va construire l'URL dynamiquement en fonction de son nom. À noter tout de même que les liens href « classiques » fonctionnent quand même.

   Pour renvoyer depuis la vue liste vers la vue détail, on va donc écrire notre lien comme ceci pour chaque livre :

```
<a href="{{ path('app_livre_detail', { id: id }) }}">
    Voir le détail
</a>
```

9. Dans la vue détail, écrire un lien qui renvoie vers la vue liste en suivant l'exemple ci-dessus. Il faudra reprendre le nom de la route qui emmène vers le template livre/index.html.twig

Maintenant qu'on a bien nos deux templates qui communiquent, essayons de rendre ça beau tant qu'à faire

Sass… ?

Normalement c'est le moment où on installe SASS et où on fait comme on a toujours fait. Sauf que CSS a depuis fait quelques progrès et qu'il supporte maintenant **l'imbrication** et les **variables**… C'est pour ça que SASS n'est **pas** inclus par défaut dans Symfony.

Rassurez-vous, il existe toujours un moyen de l'installer avec composer, puis de le faire compiler à la volée dès qu'on fait une modification, mais pour l'instant on va fonctionner avec du CSS **natif**, car c'est suffisant pour 80% des cas. Et surtout que d'ouvrir un terminal pour mettre SASS en Watch Mode + un terminal pour symfony serve, j'ai peur que ça encombre vos PCs très vite, surtout qu'on va avoir besoin d'un autre terminal pour lancer la BDD potentiellement… Évitons de se rajouter des couches inutilement ☺

On va différencier le fonctionnement de Symfony et d'Angular tout de suite : Là où Angular était tout à fait capable de gérer le **scope** de chaque composant indépendamment et donc de **cloisonner** le CSS, ce n'est pas le cas ni l'objectif de Symfony.

Nous allons donc devoir serrer un petit peu la vis et appliquer une norme de nommage que l'on appelle **BEM**, (Block Element Modifier), qui consiste simplement à préfixer chaque élément que l'on souhaite styliser par le nom de l'élément auquel il appartient.

Par exemple pour styliser ma vue détail, voici ce que j'aurais dans mon template detail.html.twig :

```
{% block body %}
<div class="livre-detail">
    <a class="livre-detail__retour-lien" href="{{ path('app_livres') }}">← Retour à la
liste</a>

    <h1 class="livre-detail__titre">{{ livre.titre }}</h1>

    <div class="livre-detail__meta">
        <p><strong>Auteur :</strong> {{ livre.auteur }}</p>
        <p><strong>Année :</strong> {{ livre.annee }}</p>
        <p><strong>Genre :</strong> {{ livre.genre }}</p>
    </div>

    <div class="livre-detail__resume">
        {{ livre.resume }}
    </div>
</div>
{% endblock %}
```

Et dans mon fichier /assets/styles/app.css

```
.livre-detail { }
.livre-detail__titre { }
.livre-detail__meta { }
.livre-detail__resume { }
.livre-detail__retour-lien { }
```

On peut néanmoins adopter une stratégie permettant de fonctionner un peu comme avec Angular. On va appeler ça l'usage de **partials** : Des fichiers destinés à être importés par app.css mais ne contenant que le code qui nous intéresse. On va comme d'habitude les préfixer d'un underscore « _ ». Exemple :

```
assets/
└── styles/
    ├── app.css            ← point d'entrée, importe tout
    ├── _variables.css     ← équivalent de la charte graphique dans Angular
    ├── _layout.css        ← styles de base.html.twig
    ├── _livre-liste.css   ← styles de livre/index.html.twig
    └── _livre-detail.css  ← styles de livre/detail.html.twig
```

Et dans app.css, on importe ces **partials** comme ceci :

```
assets > styles > # app.css
/* assets/styles/app.css */
@import '_variables.css';
@import '_layout.css';
@import '_livre-detail.css';
```

app.css devient le **point d'entrée unique** de nos feuilles de style, et donc on n'a plus qu'à l'inclure dans base.html.twig comme ceci :

```
        {% block stylesheets %}
            <link rel="stylesheet" href="{{ asset('styles/app.css') }}">
        {% endblock %}
```